```
import numpy as np
```

Ej1, b, ii

```
p1 = np.array([[0.2,0.5,0.3],[0.3,0.3,0.4],[0.7,0.2,0.1]])
p1
```

```
array([[0.2, 0.5, 0.3],
       [0.3, 0.3, 0.4],
       [0.7, 0.2, 0.1]])
```

```
np.matmul(np.array([1,0,0]),np.linalg.matrix_power(p1,10))[1]
```

```
np.float64(0.34693778029999994)
```

24/28

Ej 2

a)

```
s = list(range(11))
```

```
p = np.zeros((len(s),len(s)))
for i in range(1,10):
  p[i,i-1] = 0.53
  p[i,i+1] = 0.47
p[0,0], p[10,10] = 1,1
```

7

```
print('Planteo del proceso Markoviano:')
print(f'Estados:{s}')
print(f'Matriz de transición:\n{p}')
```

```
Planteo del proceso Markoviano:
Estados:[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Matriz de transición:
[[1.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.53 0.   0.47 0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.53 0.   0.47 0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.53 0.   0.47 0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.53 0.   0.47 0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.53 0.   0.47 0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.53 0.   0.47 0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.53 0.   0.47 0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.53 0.   0.47 0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.53 0.   0.47]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   1.  ]]
```

b)

```
change = True
p_inf = p
n=0
while change:
  p_comp = p_inf
  p_inf = np.matmul(p_inf, p)
  if np.all(p_comp == p_inf):
    change = False
  n += 1
  if n>100000:
    break
print(n)
p_inf
```

```
14309
array([[1.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        ],
       [0.94509057, 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.05490943],
```

```
[0.88317143, 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 0.11682857],
[0.81334771, 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 0.18665229],
[0.73461033, 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 0.26538967],
[0.64582137, 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 0.35417863],
[0.54569765, 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 0.45430235],
[0.43279217, 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 0.56720783],
[0.30547323, 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 0.69452677],
[0.16190081, 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 0.83809919],
[0.        , 0.        , 0.        , 0.        , 0.        ,
 0.        , 0.        , 0.        , 0.        , 0.        ,
 1.        ]])
```

```
print(f'Ganancia esperada = {sum([p * np.where(p_inf[5]==p)[0][0] for p in p_inf[5]])}')
```

```
Ganancia esperada = 3.5417862975365972
```

3 No, ganancia esperada = -10*p_inf[5,0]+10*p_inf[5,10].

7 c)

```
print('La minima cantidad de fichas para tener más probabilidad de ganar que perder el juego es de 7')
print(p_inf[7])
```

```
La minima cantidad de fichas para tener más probabilidad de ganar que perder el juego es de 7
[0.43279217 0.         0.         0.         0.         0.
 0.         0.         0.         0.         0.56720783]
```

7 d)

```
s100 = list(range(101))
```

```
p100 = np.zeros((len(s100),len(s100)))
for i in range(1,100):
  p100[i,i-1] = 0.53
  p100[i,i+1] = 0.47
p100[0,0], p100[100,100] = 1,1
```

```
change = True
p_inf100 = p100
n=0
while change:
  p_comp = p_inf100
  p_inf100 = np.matmul(p_inf100, p100)
  if np.all(p_comp == p_inf100):
    change = False
  n += 1
  if n>100000:
    break
print(n)
p_inf100 = np.round(p_inf100, 5)
```

```
100001
```

```
n=0
change = False
while not change:
  if p_inf100[n][-1] > 0.5:
    change = True
  else:
```

```
    n += 1
print(f'Para tener más probabildades de ganar que perder hay que empezar con {n} fichas')
print(p_inf100[n])
```

```
Para tener más probabildades de ganar que perder hay que empezar con 95 fichas
[0.45159 0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.      0.      0.      0.      0.      0.      0.      0.
 0.      0.54841]
```

28/28

3

a)

Estados:

- Al dia = D
- Atrasado = A

S = {AA, AD, DA, DD}

```
s = np.array(['aa','ad','da','dd'])
s
```

```
array(['aa', 'ad', 'da', 'dd'], dtype='<U2')
```

```
p = np.zeros((len(s),(len(s))))
p[0,0] = 0.8
p[0,1] = 0.2
p[3,3] = 0.9
p[3,2] = 0.1
p[1,2] = 0.5
p[1,3] = 0.5
p[2,0] = 0.7
p[2,1] = 0.3
p
```

```
array([[0.8, 0.2, 0. , 0. ],
       [0. , 0. , 0.5, 0.5],
       [0.7, 0.3, 0. , 0. ],
       [0. , 0. , 0.1, 0.9]])
```

b)

El estado al inicio del semestre es DD (al día las últimas semanas), ya que el estudiante no tiene que hacer el esfuerzo por estudiar temas de otras semanas, sino solo lo que se le enseñe la semana próxima.

c)

```
inicio = np.array([0,0,0,1])
p2 = np.matmul(inicio,np.linalg.matrix_power(p,2))
p2
```

```
array([0.07, 0.03, 0.09, 0.81])
```

```
print(f'La probabilidad de que el estudiante esté atrasado luego de 1 mes de estar al día es de {p2[0]+p2[2]:.1%}')
```

```
La probabilidad de que el estudiante esté atrasado luego de 1 mes de estar al día es de 16.0%
```

d)

```
change = True
p_inf = p
n=0
while change:
  p_comp = p_inf
  p_inf = np.matmul(p_inf, p)
  if np.all(p_comp == p_inf):
    change = False
  n += 1
  if n>100000:
    break
print(n)
p_inf = np.round(p_inf, 5)
```

```
161
```

```
p_inf[0]
```

```
array([0.33333, 0.09524, 0.09524, 0.47619])
```

```
print(f'En estado estacionario, la probabilidad de estar al dia es de {p_inf[0,1]+p_inf[0,3]:.1%}, mientras que la de estar atrasado es de {
```

```
En estado estacionario, la probabilidad de estar al dia es de 57.1%, mientras que la de estar atrasado es de 42.9%
```

4

c)

```
p0 = np.array([38.7, 21.38, 15.02, 4.67, 5.23])/85
p0
```

```
array([0.45529412, 0.25152941, 0.17670588, 0.05494118, 0.06152941])
```

```
p = np.array((
    [38.24,0.1665,0.1559,0.1201,0.0175]
    ,[0.1796,20.98,0.1188,0.0914,0.0102]
    ,[0.2120,0.1104,14.5952,0.0901,0.0123]
    ,[0.1951,0.1181,0.1007,4.2468,0.0093]
    ,[0.0031,0.0167,0.0095,0.0200,5.1807]
    ))

for i in range(p.shape[0]):
  p[i] = p[i] / p[i].sum()
```

```
print('Matriz de transición:')
print(p)
```

```
Matriz de transición:
[[9.88113695e-01 4.30232558e-03 4.02842377e-03 3.10335917e-03
  4.52196382e-04]
 [8.40037418e-03 9.81290926e-01 5.55659495e-03 4.27502339e-03
  4.77081384e-04]
 [1.41145140e-02 7.35019973e-03 9.71717710e-01 5.99866844e-03
  8.18908123e-04]
 [4.17773019e-02 2.52890792e-02 2.15631692e-02 9.09379015e-01
  1.99143469e-03]
 [5.92734226e-04 3.19311663e-03 1.81644359e-03 3.82409178e-03
  9.90573614e-01]]
```

6

d)

2 años = 8 trimestres –> 8 períodos de los comprendidos por la matriz de transición

```
p8 = np.matmul(p0,np.linalg.matrix_power(p,8))
p8
```

```
array([0.46571198, 0.25194881, 0.17289503, 0.04796161, 0.06148257])
```

```
empresas = ['Claro','Movistar','Tigo','WOM','Otros']
print('Las participaciones de mercado luego de 8 trimestres serán:')
for emp, part in zip(empresas, list(p8)):
  print(f'{emp:.<20} {part:.2%}')
```

7

```
Las participaciones de mercado luego de 8 trimestres serán:
Claro..... 46.57%
Movistar..... 25.19%
Tigo..... 17.29%
WOM..... 4.80%
Otros..... 6.15%
```

e)

```
change = True
p_inf = p
n=0
while change:
  p_comp = p_inf
  p_inf = np.matmul(p_inf, p)
  if np.all(p_comp == p_inf):
    change = False
  n += 1
  if n>100000:
    break
print(n)
p_inf = np.round(p_inf, 5)
```

```
3636
```

```
p_inf[0]
```

```
array([0.50314, 0.24234, 0.15458, 0.04137, 0.05857])
```

```
print('Las participaciones de mercado estacionarias serán:')
for emp, part in zip(empresas, list(p_inf[0])):
  print(f'{emp:.<20} {part:.2%}')
```

7

```
Las participaciones de mercado estacionarias serán:
Claro..... 50.31%
Movistar..... 24.23%
Tigo..... 15.46%
WOM..... 4.14%
Otros..... 5.86%
```

Start coding or generate with AI.